Modern Cryptography and AI/ML Security

LECTURE 28

Hybrid Cryptosystem & PQC integration

IISER THIRUVANANTHAPURAM

Course ID: DSC 4110/6111 **Instructor:** Dr. Laltu Sardar **Date:** October 14, 2026

1 Hybrid Cryptosystems

1.1 Motivation and Basic Idea

A hybrid cryptosystem combines two or more cryptographic mechanisms to achieve a desired security property. In post-quantum cryptography (PQC), a hybrid system typically combines a classical cryptographic algorithm with a post-quantum algorithm. The motivation is to reduce dependence on a single cryptographic assumption during the transition from classical to quantum-resistant cryptography.

For example, elliptic-curve cryptography (ECC) is widely used for key establishment and digital signatures. However, a sufficiently powerful quantum computer running Shor's algorithm could break the underlying discrete logarithm problem. Post-quantum algorithms, such as ML-KEM and ML-DSA, are designed to resist known classical and quantum attacks, although their security still depends on the underlying assumptions and implementation.

A hybrid system can combine a classical mechanism with a post-quantum alternative so that the intended security property continues to hold if one component becomes insecure. This guarantee depends on the construction and its security analysis; merely using two algorithms does not automatically provide hybrid security.

The two principal examples are:

- **Hybrid key establishment:** Combines classical and post-quantum shared secrets to derive a common session key.
- **Hybrid digital signatures:** Combines classical and post-quantum signatures to authenticate a message.

1.2 Hybrid Key Establishment

Key establishment enables two communicating parties, Alice and Bob, to derive a common secret key over an insecure communication channel. In a hybrid key-establishment protocol, a classical key-establishment mechanism is combined with a post-quantum key-encapsulation mechanism (KEM) to reduce dependence on a single cryptographic assumption.

Consider a hybrid construction combining elliptic-curve Diffie–Hellman (ECDH) and ML-KEM. The parties independently establish two shared secrets:

$$K_C = \mathsf{ECDH}(sk_A, pk_B)$$

and

$$K_P = \mathsf{ML\text{-}KEM.Decap}(sk_B, c_P),$$

where K_C is the classical shared secret and K_P is the post-quantum shared secret. Here, c_P is the ML-KEM ciphertext generated during encapsulation. The ECDH public-key exchange and ML-KEM encapsulation are performed according to the protocol.

The two shared secrets are combined using a suitable key-derivation function (KDF):

$$K_H = \mathsf{KDF}(K_C \| K_P, \mathsf{context}),$$

where $\|$ denotes concatenation and context binds the derived key to relevant protocol information, such as algorithm identifiers and session-specific transcript data. Both parties independently derive the same key, provided that the component exchanges succeed and the same inputs and context are used. The resulting session key can be used with an authenticated encryption scheme, such as AES-GCM or ChaCha20-Poly1305, to protect application data.

Why combine classical and post-quantum mechanisms? Hybrid key establishment reduces dependence on a single cryptographic assumption. ECDH relies on the elliptic-curve discrete logarithm problem, which Shor's algorithm can solve using a sufficiently powerful quantum computer, whereas ML-KEM relies on lattice-based assumptions believed to resist known quantum attacks. If ECDH is broken but ML-KEM remains secure, a properly designed hybrid combiner can preserve session-key security. Conversely, ECDH may provide protection if ML-KEM becomes vulnerable to an attack that does not also break ECDH. Thus, hybrid cryptography hedges against component-specific failures, but cannot protect against an adversary who breaks both components. ML-KEM alone may suffice when post-quantum security is the sole objective and its security assumptions are accepted. In either case, security requires a sound combiner, correct transcript binding, secure implementations, and appropriate failure handling, without silently falling back to classical-only key establishment.

Application in TLS 1.3. TLS 1.3 establishes session keys through a handshake that may use ephemeral Diffie–Hellman key exchange, including X25519 or standardized elliptic-curve groups. To support post-quantum security during migration, a hybrid key-establishment mechanism can combine X25519 with ML-KEM-768. The resulting shared secrets are processed according to the specified hybrid construction and TLS key schedule to derive handshake and application traffic secrets. The negotiated algorithms and exchanged values must be bound to the handshake transcript to prevent tampering and downgrade attacks. This approach allows TLS deployments to obtain post-quantum protection while retaining a classical security component, subject to the security properties of the selected hybrid construction and its implementation.

1.3 Hybrid Digital Signatures

Digital signatures provide message authentication and integrity by allowing a verifier to check whether a message was signed using the corresponding private key. A hybrid digital signature combines a classical signature scheme with a post-quantum signature scheme to reduce dependence on a single cryptographic assumption.

Suppose Alice uses ECDSA as the classical signature algorithm and ML-DSA as the post-quantum signature algorithm. For a message m, she computes

$$\sigma_C = \mathsf{Sign}_C(sk_C, m)$$

and

$$\sigma_P = \mathsf{Sign}_P(sk_P, m),$$

where sk_C and sk_P are the corresponding private keys. The hybrid signature is represented as

$$\sigma_H = (\sigma_C, \sigma_P).$$

Bob receives the message and both signatures. Under an *AND verification policy*, he accepts the message only if both signatures verify:

$$\begin{aligned}\mathsf{Verify}_H(m, \sigma_H) = 1 \quad \Longleftrightarrow \quad & \mathsf{Verify}_C(pk_C, m, \sigma_C) = 1 \\ & \wedge\, \mathsf{Verify}_P(pk_P, m, \sigma_P) = 1.\end{aligned}$$

The public keys must be securely associated with Alice's identity, and the signatures must be bound to the same message and relevant protocol context.

Under suitable composition assumptions, this construction can preserve unforgeability if at least one component signature scheme remains unforgeable. In particular, an adversary who can forge only the classical signature should still be unable to produce a valid hybrid signature without also producing a valid post-quantum signature.

An alternative is an *OR verification policy*, under which either signature is sufficient for acceptance:

$$\mathsf{Verify}_H(m, \sigma_H) = 1 \quad \Longleftrightarrow \quad \mathsf{Verify}_C(pk_C, m, \sigma_C) = 1 \ \vee\ \mathsf{Verify}_P(pk_P, m, \sigma_P) = 1.$$

Although this policy may facilitate compatibility with legacy systems, it does not provide the same protection against compromise of either component. If an attacker can forge one signature, the attacker may produce a message accepted by the verifier.

Therefore, the verification policy, public-key binding, signature encoding, and protocol context must be specified explicitly. Hybrid signatures do not automatically guarantee security merely because two signatures are generated.

Example: Hybrid Signatures in TLS. In TLS 1.3, digital signatures are used to authenticate the handshake when certificate-based authentication is employed. A server may use a signature scheme such as ECDSA to authenticate its handshake transcript. To obtain post-quantum authentication, a deployment may instead use a post-quantum signature scheme such as ML-DSA, provided that the relevant TLS implementation and certificate infrastructure support it.

A hybrid authentication design can combine a classical signature, such as ECDSA, with a post-quantum signature, such as ML-DSA, over the appropriate handshake transcript. The client must validate both signatures under the defined hybrid policy and verify that the corresponding public keys are correctly associated with the server's identity. The precise encoding, negotiation, and certificate requirements depend on the hybrid-signature protocol or profile being deployed.

It is important to distinguish hybrid authentication from hybrid key establishment in TLS. For example, combining X25519 with ML-KEM provides hybrid key establishment, whereas combining ECDSA with ML-DSA provides hybrid signature authentication. These mechanisms address different security objectives and may be deployed separately or together.

1.4 Hybrid Cryptography in Practice

Hybrid cryptography is useful during the migration of existing systems to post-quantum security. A secure communication system may combine X25519 with ML-KEM for session-key establishment and use classical, post-quantum, or hybrid signatures for authentication, depending on its security requirements and protocol support.

In a public-key infrastructure (PKI), hybrid deployment may involve multiple public keys, hybrid signatures, certificate extensions, or updated certificate-validation procedures. These changes must be supported by the relevant protocols and applications. Simply placing two public keys in a certificate does not automatically create a secure hybrid PKI.

Hybrid cryptography also introduces practical costs. Using multiple mechanisms can increase computational requirements, communication overhead, certificate sizes, and implementation complexity. Furthermore, algorithm negotiation and failure handling must prevent an attacker from forcing the parties to use a weaker configuration.

It is important to distinguish hybrid cryptography from the broader process of PQC integration. Hybrid cryptography combines multiple cryptographic mechanisms, whereas PQC integration includes hybrid deployment, direct replacement of classical algorithms with post-quantum alternatives, protocol modifications, and migration of cryptographic infrastructure.

2 Post-Quantum Cryptography (PQC) Integration

Post-Quantum Cryptography (PQC) integration refers to the process of incorporating quantum-resistant cryptographic algorithms into existing applications, communication protocols, and security infrastructures. The objective is to protect systems against potential quantum attacks while maintaining interoperability, performance, and security. Unlike hybrid cryptography, which combines multiple cryptographic mechanisms, PQC integration is a broader process that includes algorithm replacement, hybrid deployment, protocol modification, and cryptographic infrastructure migration.

2.1 Approaches to PQC Integration

PQC can be integrated into existing systems using the following approaches.

1. Direct Replacement. In this approach, a classical cryptographic algorithm is replaced with a post-quantum alternative. For example, an existing key establishment mechanism based on elliptic-curve cryptography can be replaced with ML-KEM, standardized in NIST FIPS 203. Similarly, classical digital signatures can be replaced with ML-DSA (FIPS 204) or SLH-DSA (FIPS 205). Although direct replacement simplifies the long-term cryptographic architecture, it may require modifications to protocols, certificates, libraries, and applications.

2. Hybrid Integration. In this approach, classical and post-quantum mechanisms are used together during the migration period. For example, a key-establishment protocol may combine elliptic-curve Diffie–Hellman (ECDH) with ML-KEM and derive a session key from both shared secrets using an appropriate key-derivation function. The objective is to retain security if one component becomes vulnerable, subject to the security properties of the combiner and the surrounding protocol. Hybrid integration can facilitate gradual migration while reducing dependence on a single cryptographic assumption.

3. Protocol and Infrastructure Integration. PQC integration also requires changes to existing protocols and security infrastructure. For example, integrating PQC into TLS requires support for post-quantum or hybrid key establishment, appropriate algorithm negotiation, and correct transcript binding. Similarly, migrating a public-key infrastructure (PKI) may require updates to certificate formats, signature algorithms, trust stores, and certificate-validation procedures. These changes must preserve authentication, interoperability, and the intended security properties.

2.2 Challenges in PQC Integration

Several practical challenges arise when deploying PQC:

- **Performance and communication overhead:** Post-quantum algorithms may have different computational costs, public-key sizes, ciphertext sizes, and signature sizes compared with classical algorithms.
- **Compatibility:** Existing protocols, devices, libraries, and applications may not support the required post-quantum algorithms or message sizes.
- **Security and implementation:** Integration must address secure key management, implementation vulnerabilities, algorithm negotiation, and downgrade attacks.
- **Cryptographic agility:** Systems should support controlled algorithm updates so that cryptographic mechanisms can be replaced or upgraded without redesigning the entire infrastructure.
- **Migration and interoperability:** Organizations must identify cryptographic dependencies, evaluate affected systems, test implementations, and establish a migration plan that accounts for legacy components.

2.3 Example: Integrating PQC into Secure Communication

Consider a client-server application that currently uses ECDH to establish a session key and a classical digital signature to authenticate the server. To migrate the application toward quantum-resistant security, the system can adopt either of two strategies.

In the direct-replacement approach, the key-establishment mechanism is replaced with ML-KEM, and the classical signature mechanism is replaced with an appropriate post-quantum signature scheme.

Alternatively, the system can initially use hybrid key establishment by combining ECDH with ML-KEM. It can also deploy an appropriate hybrid signature construction if both classical and post-quantum authentication are required. After interoperability testing and policy evaluation, the organization may transition to a post-quantum-only configuration.

The migration must ensure that the selected algorithms are correctly negotiated, the resulting keys are securely derived, and authentication and failure handling remain secure throughout the process.